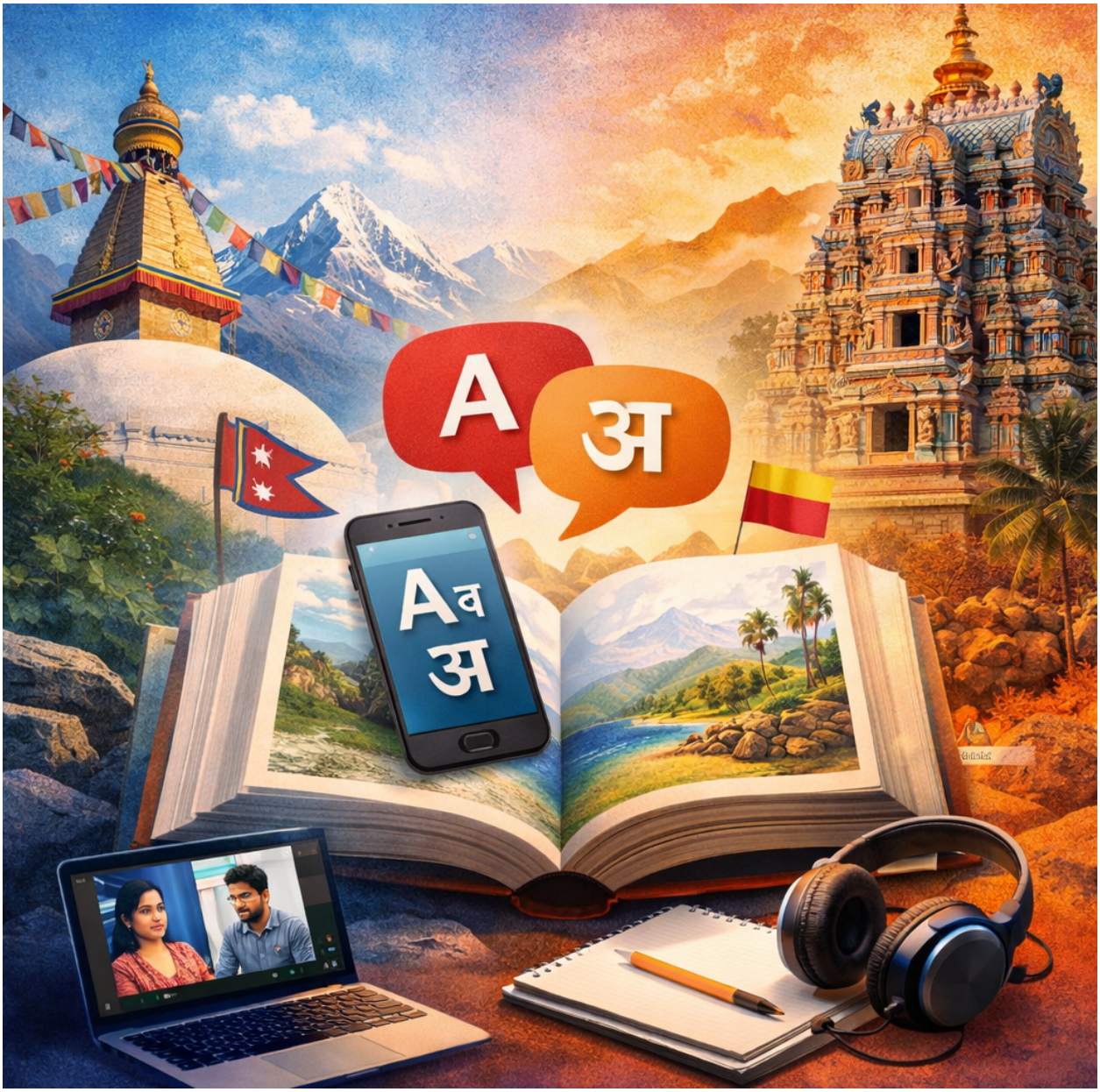
# Report - Design Rationale

Hardik Shrestha

BhashaQuest - Dual-Stream Language Learning (Nepali & Kannada)



# Contents


**1 Architecture Overview** **3**
- 1.1 System Structure . . . 3
- 1.2 Key Architectural Principles . . . 3
- 1.3 Requirements Coverage . . . 3
- 1.4 Notable Implementation Details . . . 3

**2 Modules and Responsibilities** **4**
- 2.1 UI Layer . . . 4
- 2.2 Controller Layer . . . 4
- 2.3 Domain Layer . . . 4
- 2.4 Data Layer . . . 4

**3 Data Model and Content Schema** **5**
- 3.1 Content JSON Shape . . . 5
- 3.2 Profile Persistence . . . 5

**4 Flows and Behavior** **5**
- 4.1 Unit Progression Flow . . . 5
- 4.2 UI Styling and Interactions . . . 5
- 4.3 Error Handling and Robustness . . . 6

**5 Design Patterns** **6**
- 5.1 Factory: `ExerciseFactory` (Creational) . . . 6
- 5.2 Singleton: `AudioManager` (Creational) . . . 6
- 5.3 Strategy: `StrategyGrader` Hierarchy (Behavioral) . . . 7
- 5.4 Observer: `Subject/ProfileObserver` (Behavioral) . . . 7

**6 Features and Flows** **7**
- 6.1 Unit Progression Flow . . . 8

**7 Testing** **8**
- 7.1 Coverage . . . 8
- 7.2 Summary Table . . . 8
- 7.3 Gaps / Next Tests . . . 8
- 7.4 Execution (Qt Creator AutoTest) . . . 8
- 7.5 gMock Example . . . 9

**8 Error Handling** **9**

**9 Architecture Diagrams** **10**

**10 Reflection** **13**

**11 Future Work** **13**

**12 Conclusion** **13**

# 1 Architecture Overview

## 1.1 System Structure

Bhasha Quest uses layered MVC with strict separation of UI, controller, domain, and data. The final design adds dual streams (Nepali, Kannada) and unit-based progression (3 exercises per unit) surfaced in a Path tab. A dedicated Path view surfaces status and locking; the controller orchestrates grading, progression, and persistence via repositories.

## 1.2 Key Architectural Principles

- **Strict Layering:** UI → Controller → Domain → Data; domain has no UI knowledge.
- **Streams & Units:** Content organized into streams with units of 3 exercises; Path tab shows completed/current/locked units with admin override.
- **Controller as Orchestrator:** `AppController` holds stream/unit context, picks graders at runtime, updates profile/XP/streak, persists via repositories, emits unit completion.
- **Signals/Slots for Decoupling:** UI emits actions; controller emits state/results/errors; keeps dependencies one-way.
- **Repository Abstraction:** JSON repositories for content and profile; SQLite profile repo behind the same interface for future backend swap.
- **Error Handling:** Controller surfaces repo/audio issues via signals; Path respects locks unless admin override (admin123).

## 1.3 Requirements Coverage

- **Dual Languages:** Two parallel learning streams (Nepali, Kannada) with independent units and content.
- **Structured Units:** Each unit holds exactly 3 exercises; Path tab visualizes progress and locking.
- **Exercise Variety:** MCQ, Translate (text/character selection), TileOrder, Listen (audio MCQ), Speak (stub evaluator); audio integrated.
- **Progress Tracking:** XP, streak, skill mastery, per-stream unit completion persisted; UI shows date and streak controls.
- **Patterns Implemented:** Factory + Singleton (creational), Strategy + Observer (behavioral), all in active use.
- **Testing:** GoogleTest/GoogleMock harness; core logic tests included and passing.

## 1.4 Notable Implementation Details

- **Path Locking & Admin Override:** Units unlock sequentially; admin password can bypass locks for demos/testing.
- **Unit Completion Dialog:** Confetti animation + "Keep Learning" action wired to controller continuation.
- **Character Picker:** Per-language character sets flow from content JSON to the widget, which rebuilds its grid dynamically.
- **Audio Resolution:** AudioManager resolves relative paths under `assets/`; missing files are safely ignored.
- **Persistence Shape:** Profile JSON stores completed units; SQLite schema includes unit progress table for future DB promotion.

# 2 Modules and Responsibilities

## 2.1 UI Layer

- **HomeView:** Stream cards, profile shortcut.
- **LessonView:** Prompts, MCQ list (click-to-fill), character picker, audio play, unit progress label, feedback.
- **PathView:** Unit list per stream with status; prompts for admin password on locked units.
- **ProfileView:** Learner Hardik, XP, streak, skills; +1 day button for streak testing.
- **UnitCompleteDialog:** Lightweight animation and continue action.

## 2.2 Controller Layer

- **AppController:** Loads streams/units, tracks indices, selects graders (Strategy), updates profile (Observer), persists via repository, emits unitCompleted/streamFinished/unitProgress/errors.

## 2.3 Domain Layer

- **Exercises:** MCQ, Translate (character selection), TileOrder, Listen, Speak stub.
- **Graders (Strategy):** MCQ/Translate/CharacterSelection/TileOrder/Listen/Speak.
- **Progress:** Profile (Subject/Observer), SkillProgress, SRSScheduler.
- **Creational:** ExerciseFactory (Factory), AudioManager (Singleton).
- **Structure:** LearningStream, Unit (vector of unique_ptr¡Exercise¿).

## 2.4 Data Layer

- **Repositories:** JsonContentRepository, JsonProfileRepository, SQLiteProfileRepository (schema-ready).
- **Assets:** Audio files under `code/assets/audio`; content under `code/core/data/content.json`.

# 3 Data Model and Content Schema

## 3.1 Content JSON Shape

```
{
  "streams": [
    {
      "id": "nepali",
      "name": "Learn Nepali",
      "language": "Nepali",
      "units": [
        {
          "id": "nepali-greetings-1",
          "title": "Greetings 1",
          "exercises": [
            { "type": "MCQ", "spec": { "id": "...", "prompt": "...",
                "choices": [...], "correctIndex": 0, "audioPath": "assets/audio/..." }},
            { "type": "Translate", "spec": { "characterSelection": true,
                "characterSet": [...], "answers": [...] }},
            { "type": "TileOrder", "spec": { "tiles": [...], "correctOrder": [...] }}
          ]
        }
      ]
    }
  ]
}
```

Listing 1: Stream/unit content schema (excerpt)

## 3.2 Profile Persistence

- **JSON:** Stores name, xp, streak, skills (mastery/attempts), completedUnits per stream.
- **SQLite:** Tables for profile, skill_progress, unit_progress (future DB promotion).

# 4 Flows and Behavior

## 4.1 Unit Progression Flow

1. User selects stream/unit (Home/Path); controller loads streams and sets indices.
2. LessonView shows exercise; user submits; controller grades (Strategy), updates profile (Observer), plays audio (Singleton).
3. After last exercise: controller marks unit complete, persists profile, emits `unitCompleted`; UI shows dialog and continues on user action.
4. Path reflects completion; next unit unlocks; admin override can bypass lock.

## 4.2 UI Styling and Interactions

- LessonView: progress label, click-to-select MCQ, character picker for selection mode, audio button visible when audioPath exists.
- PathView: dual columns (Nepali/Kannada), locked items greyed; admin prompt on locked double-click.
- ProfileView: hardcoded learner “Hardik”; streak/date control for testing.
- Dialog: confetti animation and CTA to continue.

### 4.3 Error Handling and Robustness

- Missing content/profile file: controller emits error; UI message box.
- Missing audio file: AudioManager quietly ignores; no crash.
- Bounds checks: controller guards stream/unit/exercise indices and emits errors on invalid state.
- Path locking: enforced by completion; admin override requires password.
- Future: stronger JSON validation, retries, and logging.

## 5 Design Patterns

### 5.1 Factory: `ExerciseFactory` (Creational)

**Location:** `code/core/domain/ExerciseFactory.h/cpp`.
**Why:** Centralizes creation of MCQ, Translate, TileOrder, Listen, Speak from JSON; avoids conditional sprawl; preserves Open/Closed.
**How:** `createExercise(type,spec)` dispatches to type-specific builders; supports audio paths, character sets, selection flags.
**Benefit:** Localized construction, easy extension, tested in isolation.
**Snippet:**

```
Exercise* ExerciseFactory::createExercise(const QString& type,
                                          const QJsonObject& spec) {
    if (type == "MCQ") return createMCQ(spec);
    if (type == "Translate") return createTranslate(spec);
    if (type == "TileOrder") return createTileOrder(spec);
    if (type == "Listen") return createListen(spec);
    if (type == "Speak") return createSpeak(spec);
    return nullptr;
}
```

Listing 2: Factory dispatch

### 5.2 Singleton: `AudioManager` (Creational)

**Location:** `code/core/domain/AudioManager.h/cpp`.
**Why:** Single `QMediaPlayer`/`QAudioOutput` to avoid resource contention and duplicate init.
**How:** Meyers singleton; `playAudio()`, `playSuccess()`, `playError()`; deleted copy/move.
**Benefit:** Controlled audio lifecycle; callable from controller/UI without extra players.

### 5.3 Strategy: `StrategyGrader` Hierarchy (Behavioral)

**Location:** `code/core/domain/StrategyGrader.h` and concrete graders.
**Why:** Different grading algorithms per exercise type; keeps Exercise/AppController slim.
**How:** Abstract `grade()`; concrete MCQ/Translate/TileOrder/Listen/Speak/CharacterSelection; runtime selection in `AppController`.
**Benefit:** Swap/extend grading without touching controllers; unit-tested.
**Snippet:**

```
std::unique_ptr<StrategyGrader> AppController::createGraderFor(
    const Exercise* ex) const {
    if (ex->type() == "MCQ") return std::make_unique<MCQGrader>();
    if (ex->type() == "Translate") {
        auto tr = dynamic_cast<const TranslateExercise*>(ex);
        if (tr && tr->usesCharacterSelection())
            return std::make_unique<CharacterSelectionGrader>();
        return std::make_unique<TranslateGrader>();
    }
    if (ex->type() == "TileOrder") return std::make_unique<TileOrderGrader>();
    if (ex->type() == "Listen") return std::make_unique<ListenGrader>();
    if (ex->type() == "Speak") return std::make_unique<SpeakGrader>();
    return nullptr;
}
```

Listing 3: Runtime strategy selection

### 5.4 Observer: `Subject/ProfileObserver` (Behavioral)

**Location:** `code/core/domain/Subject.h`, `Profile.h`.
**Why:** Push XP/streak/skill/unit updates to UI without tight coupling.
**How:** Profile inherits Subject; ProfileView attaches; notifications on profile changes.
**Benefit:** UI stays in sync; supports future observers (analytics, badges).

## 6 Features and Flows

- **Dual Streams:** Nepali and Kannada, each with structured units of 3 exercises.
- **Path Tab:** Shows unit status (completed/current/locked); admin override (admin123) to jump ahead.
- **Unit Completion:** Controller marks completion, persists profile, emits signal; UI shows confetti dialog with "Keep Learning."
- **Exercises:** MCQ (click-to-fill), Translate (text or character selection with per-language sets), TileOrder, Listen (audio + MCQ), Speak (stub evaluator).
- **Audio:** Centralized via `AudioManager`; plays success/error and exercise audio paths.
- **Profile:** Tracks XP, streak, skills, and completed units per stream; date advancement button for streak testing (UI).

### 6.1 Unit Progression Flow

1. User selects stream/unit (Home or Path). Controller loads stream and sets unit/exercise index.
2. LessonView shows exercise; user submits answer; controller grades via Strategy.
3. Profile updated (XP, streak, skill attempt); profile persisted.
4. After final exercise in unit: controller marks unit complete, emits `unitCompleted`; UI shows dialog and continues on user request.
5. Path reflects completed unit; next unit unlocks (or admin override bypasses lock).

## 7 Testing

### 7.1 Coverage

- ExerciseFactory: creation of MCQ/Translate/TileOrder/Listen/Speak; audio/character sets; unknown type returns null.
- Graders: MCQ/Translate/CharacterSelection/TileOrder/Listen/Speak stub correctness paths.
- SRSScheduler: Easy/Medium/Hard intervals.
- Profile: XP, streak reset, unit completion tracking.
- JsonContentRepository: loads streams/units from fixture; missing file error handling.

### 7.2 Summary Table

| **Suite** | **Focus** |
|---|---|
| ExerciseFactory | Type dispatch; audio/character specs; unknown types |
| Graders | MCQ/Translate/CharacterSelection/TileOrder/Listen/Speak correctness |
| SRSScheduler | Interval calculation (Easy/Medium/Hard) |
| Profile | XP/streak updates; unit completion map |
| ContentRepository | Streams/units load; missing file error path |
| Controller (planned) | Signal flow with gmock repos; malformed content |

### 7.3 Gaps / Next Tests

Controller signal flow with mocks; edge cases for malformed content; more exhaustive speak/listen scoring cases; profile persistence round-trip.

### 7.4 Execution (Qt Creator AutoTest)

- Open `code/tests/tests.pro`; build target `bhashaquest_tests`.
- Run via AutoTest pane (GoogleTest framework enabled) or CLI as above.
- All current tests pass on latest build; no known flakiness.

### 7.5 gMock Example

Mocking `IContentRepository` to validate controller startup:

```
class MockContentRepo : public IContentRepository {
public:
    MOCK_METHOD(std::vector<LearningStream>, loadStreams, (QString*), (override));
};

TEST(ControllerTest, StartsStreamWithMockRepo) {
    MockContentRepo repo;
    JsonProfileRepository profileRepo(":/tmp/profile.json");
    AppController controller(&repo, &profileRepo);
    std::vector<LearningStream> streams;
    LearningStream s("s1", "Stream 1", "L");
    Unit u("u1", "Unit 1");
    u.addExercise(std::make_unique<MCQExercise>("e1","p",{ "a","b" },0));
    s.addUnit(std::move(u));
    streams.push_back(std::move(s));

    EXPECT_CALL(repo, loadStreams(testing::_))
        .WillOnce(testing::Return(streams));

    controller.initialize();
    ASSERT_TRUE(controller.startStream("s1"));
    EXPECT_NE(controller.currentExercise(), nullptr);
}
```

Listing 4: EXPECT_CALL example

# 8 Error Handling

- Repo load/save errors surfaced via controller signal; UI shows message box.
- Missing audio: `AudioManager` no-ops if file absent.
- Path locking: enforces sequential units; admin override allowed via password.
- Controller guards: checks bounds on stream/unit/exercise indices and emits errors on invalid state.
- Content assumptions: minimal validation; future work includes stricter JSON schema and fallback defaults.

# 9 Architecture Diagrams

The diagrams below are aligned with the current codebase and highlight layer boundaries, pattern placements, and orchestration paths.

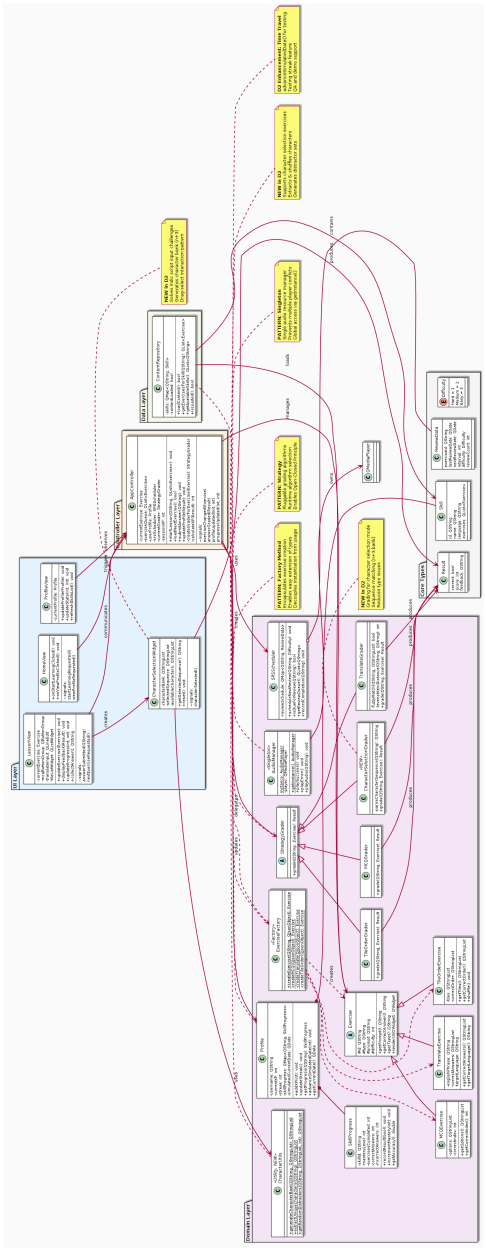


Figure 1: Class diagram showing strict UI → Controller → Domain → Data layering, Factory/Strategy/Singleton placements, and new Path/unit constructs.

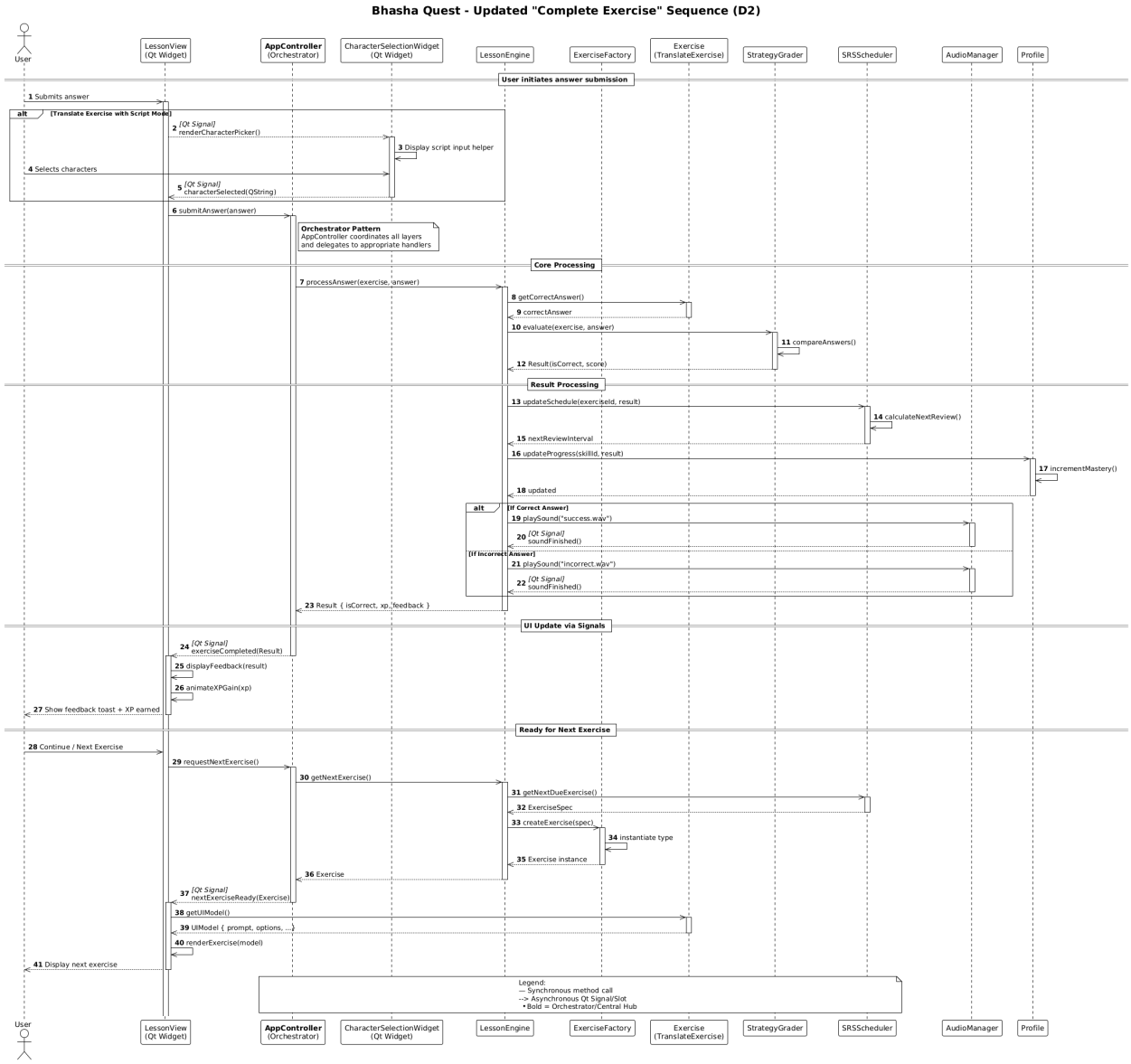



Figure 2: Sequence diagram for a complete exercise submission through unit completion, including controller orchestration and audio/feedback.

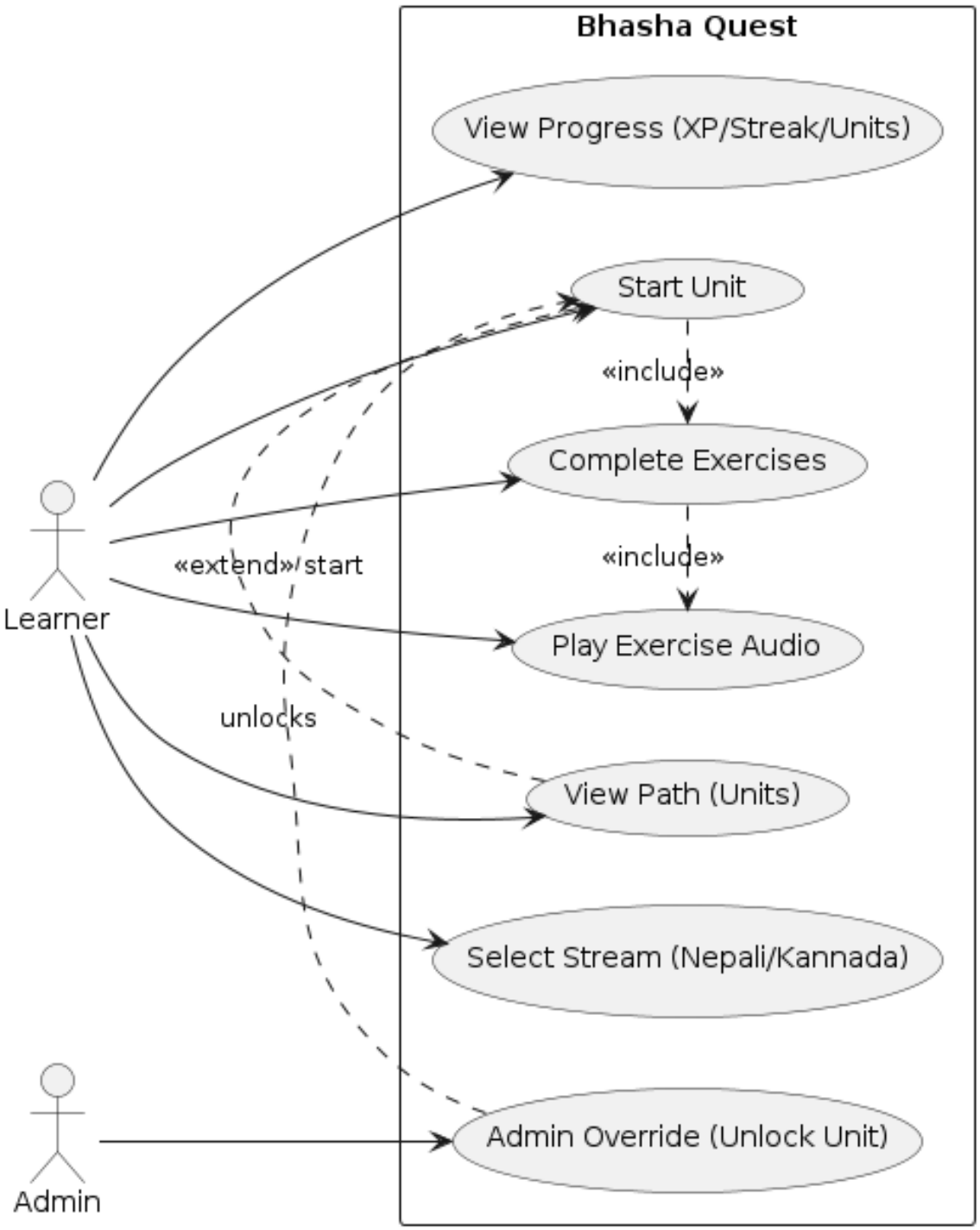



Figure 3: Use case diagram covering learner paths (Nepali/Kannada), admin override, profile viewing, and unit completion celebration.

# 10 Reflection

The layered architecture and pattern choices ultimately made adding streams, units, and new exercise types straightforward, but arriving at this structure was iterative rather than obvious from the start. Early development surfaced a tension between breadth of features and end-to-end usability, which required repeatedly narrowing scope to stabilize a complete learning loop. Strategy kept grading isolated, Factory simplified content loading, and Observer kept the UI in sync; together, these decisions reduced rework as requirements evolved. As the system grew, the main risks became clearer. In particular, encountering ambiguous UI failures highlighted that functional correctness alone was insufficient without graceful failure modes. Extending gmock-based controller tests and adding persistence round-trips would further strengthen robustness. The dual-stream Listen/Speak structure reflects a deliberate balance between coherence and extensibility, leaving room for future language support without destabilizing the core system.

Lessons learned:

- Prototype-to-architecture payoff: building a small end-to-end slice early clarified where real complexity lived and reduced controller churn when Listen/Speak requirements shifted.
- Data-driven content (JSON) reduced friction during iteration and enabled safer expansion, while forcing clearer decisions around per-language character sets.
- Testing built regression confidence, but integration gaps revealed the need to move coverage closer to controllers and mocks for stronger guarantees.
- UI resilience emerged as a trust issue rather than a cosmetic one; clearer error surfaces and loading states are essential to robustness.
- Documentation cadence matters: keeping diagrams and code aligned early reduced later decision friction and prevented unnecessary churn.
- Scope discipline required tradeoffs: stubbing Speak kept the project shippable and prevented audio handling from dominating development; future iterations should harden capture, permissions, and error handling once the core loop is stable.

# 11 Future Work

- Replace Speak stub with real audio capture + ASR integration; richer Listen content.
- Expand tests with gmock for controller/repo interactions and malformed content coverage.
- Promote SQLite persistence to primary backend; add migrations and history tables.
- Improve Path/Unit UX (summaries, streak indicators) and enhance error feedback.

# 12 Conclusion

The final system delivers dual-language learning with structured units, clear progression via the Path tab, and integrated patterns (Factory, Singleton, Strategy, Observer) underpinning a layered architecture. Core logic is covered by GoogleTest; the design is extensible for additional languages, deeper audio, and stronger persistence while preserving clean separation of concerns.